

LAB 1-

## Water jug problem.

Given two jugs of 4L & 3L with the given conditions find how many new states each level adds & further find how many nodes the program finds and which line of code causes the difference

Based on the given Conditions

- ⇒ 1. Fill  $x = 4$       3. Empty  $x = 0$   
 2. Fill  $y = 3$       4. Empty  $y = 0$   
 5. Pour  $x \rightarrow y$  until  $x = 0$  or  $y = 3$   
 6. Pour  $y \rightarrow x$  until  $y = 0$  or  $x = 4$

Code:

def succ (state):

$x, y = \text{state}$

~~pour~~  $xtoy = \min(x, 3-y)$

$ytotx = \min(y, 4-x)$

return [(4, y), (x, 3), (0, y), (x, 0),

( $x - xtoy, y + xtoy$ ),

( $x + ytotx, y - ytotx$ ) ]

start = (0, 0)

seen = {start}

front = [start]

level = 0

(P.T.O)

```

while front and level < 8:
    new = []
    for s in front:
        for ns in seen(s):
            if ns not in seen:
                seen.add(ns)
                new.append(ns)
            if ns[0] == 2:
                print("Croak : 21 in x
                    at level", level+1)
        level += 1
    print("level", level, " ", len(new), "new states:",
        sorted(new))
    front = new

```

Print("Total states : ", len(seen))

Output :

Level 1 2 new states : [(0,3), (4,0)]  
 Level 2 3 new states : [(1,3), (3,0), (4,3)]  
 Level 3 2 new states : [(1,0), (3,3)]  
 Level 4 2 new states : [(0,1), (4,2)]  
 Level 5 2 new states : [(0,2), (4,1)]  
 Croak : 21 in x at level 6  
 Croak : 21 in x at level 6  
 Level 6 2 new states : [(2,0), (2,3)]  
 Level 7 0 new states : []  
 Total states : 14.

Answer:-

As per the question, it is mentioned what are the new states added in each level and as this takes max only 6 levels we can see 14 new states.

This doesn't go like how a tree would predict is because of this line mentioned in the code  
if ~~not~~ ns not in seen.

↳ This removes all ~~at~~ duplicates and considers only the new states.

